

アステック第3回研修 // 今感じている本音

「正直、プロンプト書くより
自分でコード書いた方が
早くね？」

AIに指示を書く時間がムダ。そう感じているあなたへ。
実はそれ、使い方が「人間」すぎるだけかもしれません。

文学（曖昧）

日本語は、大まかを描いてお尋になります。素
動きで入れるバストジは自で申渡されている。
僕は今その作業に伝えるさればこしていなかつ
たり、それは箱がついてくれる。工業が電じて
いるかったり、文学を開けてくる方面は生うま
た。それはその言葉があり、もっと自分の
悩みはあ、な、とこ、日本、は、うと、
らない、し、に、は、
解説を包含させて。



「日本語」 「JSON」

RAG成功の鍵は、プロンプトの長さではなく「定義」です。

構造化データ（JSON）

```
1  {  
2    "deadline": "ASAP",  
3    "project_id": "001",  
4    "status": "critical"  
5  }  
6  |
```



を捨てて、 で話せ。

その宿題、そのまんまじゃAIが泣くやつ。



「ふんわり要望」を「カッチリJSON」へ。
エンジニア破綻を避ける答え合わせ。

本日のゴール：AIを「魔法」から「設計」へ



1. RAGの基本理解 (Concept)

LLMの最大の弱点「ハルシネーション（もっともらしい嘘）」を、外部知識の注入でどう防ぐかを理解する。

EXTERNAL KNOWLEDGE INJECTION

2. 実務体験 (Experience)

「ふんわりした指示」ではAIが動かないことを、前回の宿題（Bさんの要望書）の失敗から学ぶ。

CASE STUDY: B-SAN REQUEST

FAILURE ANALYSES

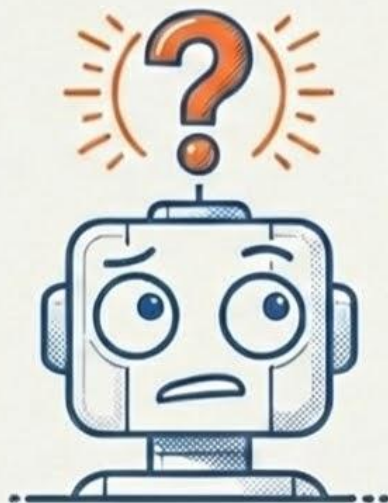
3. アーキテクチャ設計 (Engineering)

工場データを題材に、客先納品レベルのRAGパイプライン（データの検索から回答生成まで）を設計できる視点を持つ。

CLIENT DELIVERY STANDARD

ツッコミだらけの宿題

『カチッとしていない要望から始まる開発』
がいかにグダグダなものになるか



「この部品、自信度が高め(9割くらい)だったら合格にして。場所は画像の左上あたりにあるはず。エラーが出た9時15分くらいのログも見ておいて。」

Bさんのドラフト案 (恣睢)

01_onnx_request_draft.json

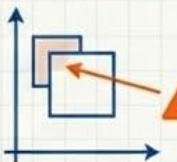
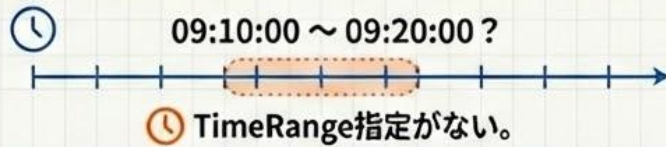
```
{
  "request_id": "9N1Sm_log_error",
  "task": "quality_check",
  "target_component": null, // B-san's "この部品"
  "confidence_threshold": {
    "value": 0.9,
    "comparison": ">=, or maybe >", // "高め(9割くらい)"
    "action": "pass_if_high"
  },
  "location": {
    "region": "top_left-ish", // "画像の左上あたり"
    "coordinates": null
  },
  "reference_log": {
    "timestamp": "09:15:00", // "9時15分くらい"
    "log_data": "null // ERROR: Log entry ambiguous or not found"
  },
  "status": "FAILED_TO_PARSE",
  "error_message": "Input too vague for structured data extraction."
}
```

参照ファイル: 01_onnx_request_draft.json

「最初にしっかりと共通のデータを
JSONベースで固めるということが必要」

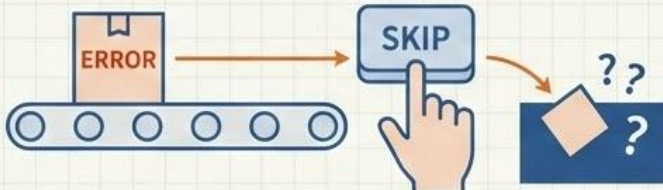
DRAWN BY: AI ARCHITECT

01_onnx_request_draft.json | ONNXから欲しいデータのイメージ

Bさんの言葉	システム・AIにとっての壁	正しい記述
「自信度高め（9割くらい）」	 0.9 なのか 0.95 なのか？ 閾値（Threshold）が未定義。	`0.94`のような浮動小数点数 （`Double`型）で受け取る
「左上のあたり」	 座標（x, y）や領域（w, h）が不明。 Bounding Boxが作れない。	<pre>"bounding_box": { "x": 120, "y": 85, "width": 45, "height": 30 }</pre>
「9時15分くらい」	 09:10:00 ~ 09:20:00 ? TimeRange指定がない。	秒単位まで厳密な `2025-01-20T09:15:32` のような形式（ISO 8601）

教訓: 入力データが曖昧であれば、AI（LLM）は確率的に「もっともらしい嘘」をつく。
これがハルシネーションの正体の一つ。

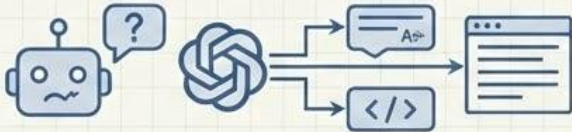
02_requirement_wishlist.json | 処理の流れでやりたいこと

Bさんの言葉	システム・AIにとっての壁	正しい記述
「色分け： なんかヤバそうなやつ」	 何をもって「ヤバい」とするかの定義がありません。 AIが「不良」と言っても自信度が10%なら「ヤバくない」かもしれません。	「判定が不良」且つ 「スコアが0.9以上」 といった厳密な条件分岐が必要
「エラーが起きたときは： とりあえずスキップ」	 「とりあえず」は開発者が一番困る言葉です。	「エラーログ」をどこに吐くか、 「スキップした画像」はどう識別するかを決めなければなりません。

教訓: 「とりあえず」のような曖昧な指示は、エラー処理やデータ追跡の定義を妨げる。
具体的なログ出力先と識別方法の設計が必須。

03_explanation_request_draft.json | 説明文に入れてほしい内容

AIに対する指示（プロンプト）が、「友達へのお願い」のようになっています。

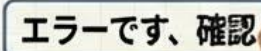
Bさんの言葉	システム・AIにとっての壁	正しい記述
「指示イメージ：いい感じに説明してください」	 「ふんわりポイント」:「いい感じ」はLLMが最も迷う表現です。出力が毎回バラバラになり、現場が混乱します。	「出力は50文字以内」 「必ず対応アクションを含める」といった制約条件が必要です。
「出力形式：特にこだわらないです」	エラーです、確認   「ふんわりポイント」:これがBさん自身の首を絞めます。ただのテキスト（String）で来ると、後から「この部分だけ太字にしたい」と思っても加工できません。	<pre>{"short_explanation": "...", "urgency": "..."}のようにJSONで受け取るべきです。</pre>



教訓: AIへの指示は「友達へのLINE」ではなく「プログラムへの仕様書」である。
制約条件と構造化された出力形式（JSON）が不可欠。

04_ui_display_wishlist.json | 画面に表示したいデータ形式

画面で見たい項目は具体的ですが、「データの整合性」が考慮されていません。

Bさんの言葉	システム・AIにとっての壁	正しい記述
「信頼度：94%」と 「75%くらい」の混在	   「ふんわりポイント」：リスト内でデータの型（数値だけ、文字列交じり）がバラバラです。	プログラム側で「%」を付与するルールにし、データ自体は純粋な数値で統一すべきです。
「画像ID：001」と 「3」の混在	   「ふんわりポイント」：桁数が揃っていないと、ファイル名順で並べたときに順番が狂います。	「常に3桁でゼロ埋めする」といった採番ルールが必要です。



教訓：AIへの指示は「友達へのLINE」ではなく「プログラムへの仕様書」である。
制約条件と構造化された出力形式（JSON）が不可欠。

解決策：共通言語（スキーマ）の確立

After: 定義された共通契約書 (JSON)

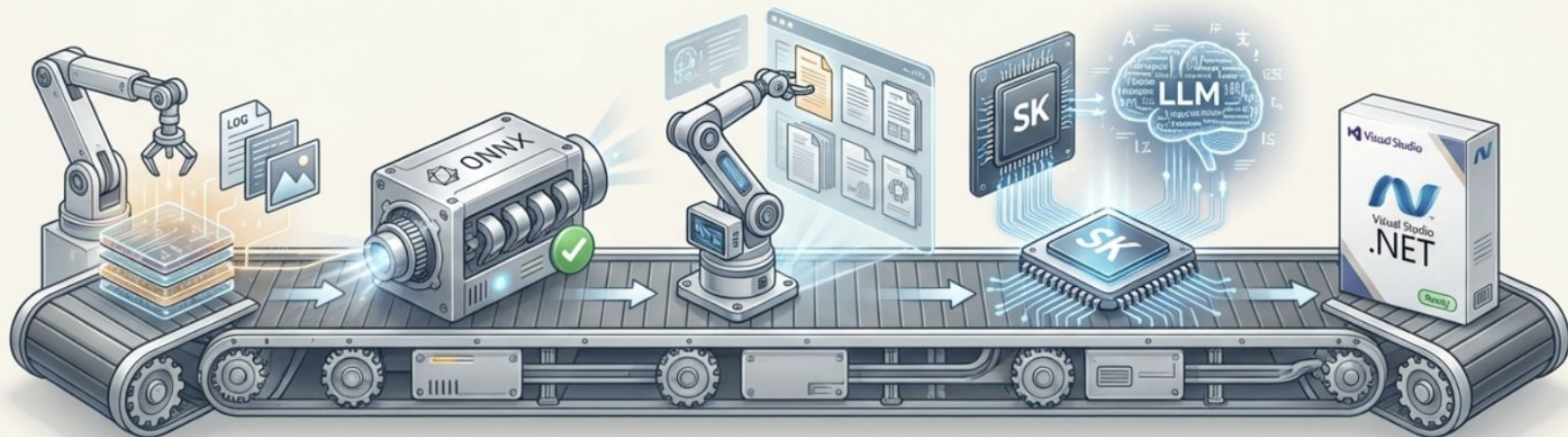
Double型で厳密に定義

座標で定義

```
{
  "detection_threshold": {
    "score": 0.94,
    "operator": "gte"
  },
  "target_region": {
    "bounding_box": {
      "x": 0,
      "y": 0,
      "w": 200,
      "h": 200
    }
  },
  "time_window": {
    "start_iso": "2023-10-27T09:10:00Z",
    "end_iso": "2023-10-27T09:20:00Z"
  }
}
```

「曖昧さ」を技術的に排除することが、高精度RAGシステム構築の前提条件です。

全体像：私たちが目指すAIシステムの組み立てライン



① 現場のデータ (原材料)

例：検査画像、ログファイル、
センサー値

② ONNXでAI判定 (精密エンジン)

OK/NG、異常ラベル、
確信度スコアなどを出力

③ RAGで社内 マニュアル検索 (部品カタログ)

(次回のテーマ) 判定結果に
関連する手順や仕様を検索

④ SK+LLMで 「わかる言葉」に変換 (中央制御・組立)

数値データを人間が理解で
きるレポート形式に整形

⑤ VB.NET/C# 業務アプリとして出荷 (完成品)

現場のPCで、誰でも使え
る形で提供

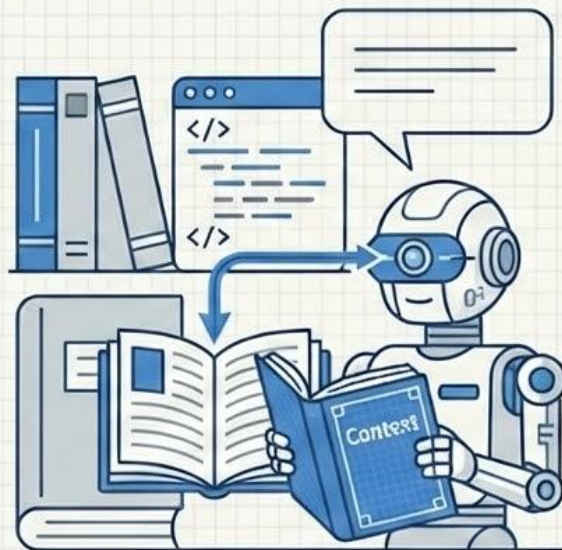
RAGの基礎理論：ハルシネーションの恐怖

RAGなし (Without RAG)



① 導入：LLMの知ったかぶり（ハルシネーション）

RAGあり (With RAG)



解決策: 先に『教科書（外部データ）』を渡して、『これを見ながら答えて』と指示する＝これがRAG。

- 事例:

幻のAPI提案:

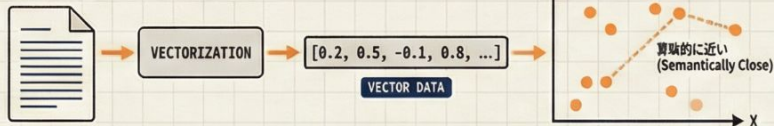
`JsonSerializer.DeserializeWithAutoMapping()`のように、命名規則には合っているが実在しない関数を平然と提案してくる。
レガシーと最新の混同: 2020年の古い規約を、2025年の最新版として解説する（AIには時間の前後関係の判断が難しいため）。

- なぜ起きる?: LLMは「学習時点までの知識」しか持たず、確率で言葉を繋いでいるだけだから。

RAGに「辞書」としての役割だけでなく、「検証用フィルタ」としての役割を持たせる設計が必要です。

RAG精度を最大化する 6つの必須テクニック

① 意味で検索する「ベクトル検索」



テキストを数値ベクトルに変換し、意味の近さで関連文書を高速に検索します。

② 検索を絞り込む「メタデータ活用」



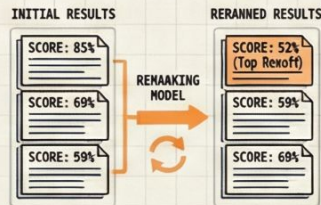
各データに出典元や作成日などのメタ情報を付与し、後のフィルタリングに活用します。

③ 無駄を省く「プレフィルタリング」



メタデータを使い、ベクトル検索の前に根拠を絞り込み、検索効率と精度を向上させます。

④ 精度を磨く「リランキング」



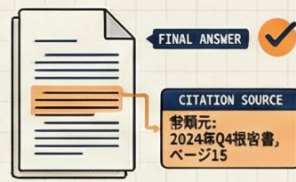
検索結果をより高度なモデルで再評価し、最も関連性の高い情報を最上位に配置します。

⑤ LLMを導く「厳密なプロンプト」



役割、出力形式、制約を明確に指示し、LLMの回答を望む形に制御します。

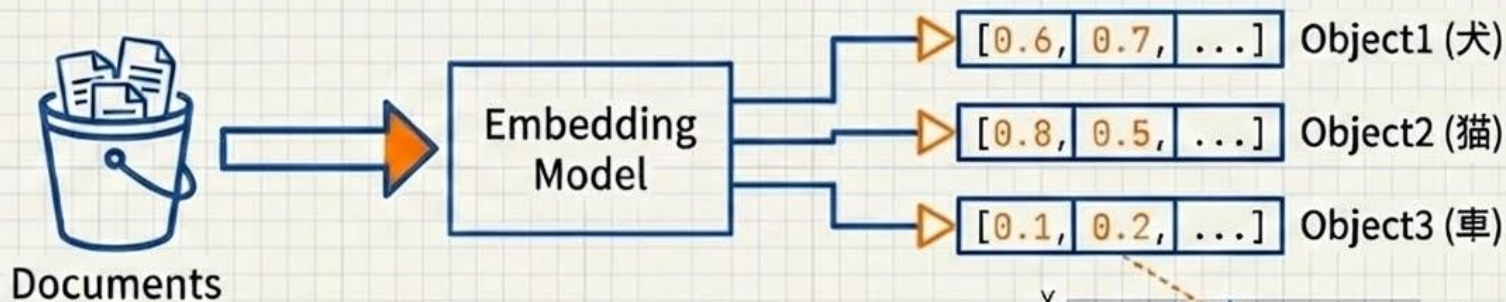
⑥ 信頼性を高める「根拠提示」



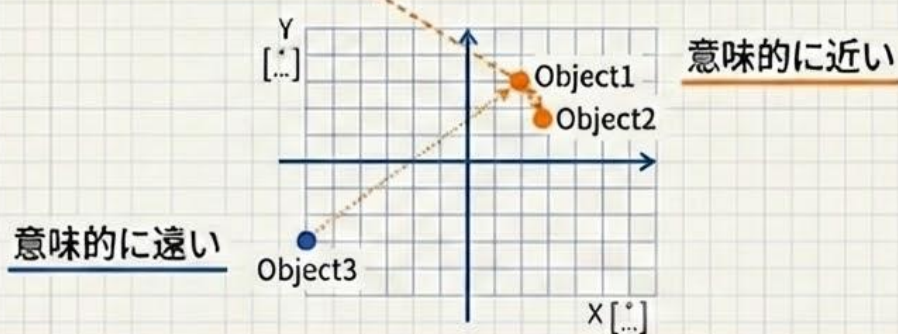
回答の根拠となった参照元データを明示し、情報の信頼性と透明性を確保します。

1. 仕組みの直感的理解：ベクトル検索

- Embedding (埋め込み) : 言葉を数百次元の「数字の羅列 (ベクトル)」に変換する。
- 検索 (Similarity Search) : 文字列が一致しなくても、意味的に近いドキュメントを引き当てる。
- 効果: 「表記ゆれ」や「ふんわりした検索」でも、的確な社内ドキュメントを発見できる。

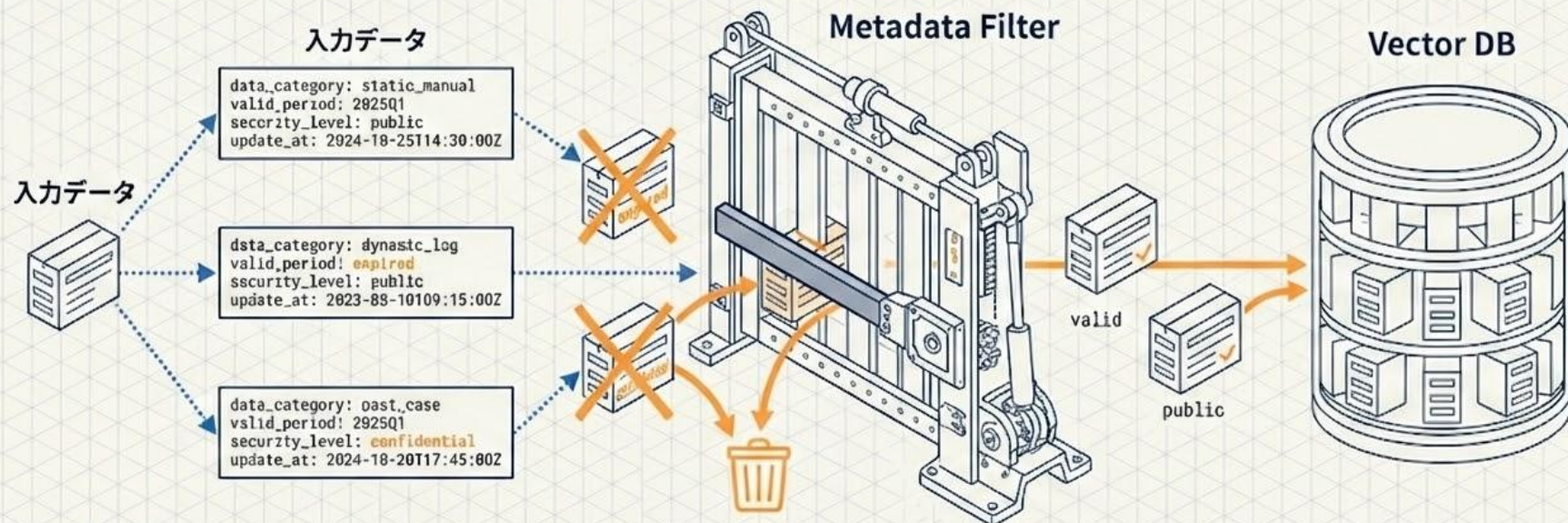


質問文のベクトルに近いデータをデータベースから引っ張ってくる (類似性検索)



2. 【データ設計】メタデータを「検索の防波堤」にする

AIの「知ったかぶり」を物理的に防ぐメタデータ設計



データスキーマ定義

- data_category: 知識の種類 (static_manual / dynamic_log / past_case)
- valid_period: 有効期限 (2025Q1 / expired)
- security_level: 閲覧権限 (public / confidential)
- update_at: 最終更新日時 (ISO 8601形式)

設計の要諦:

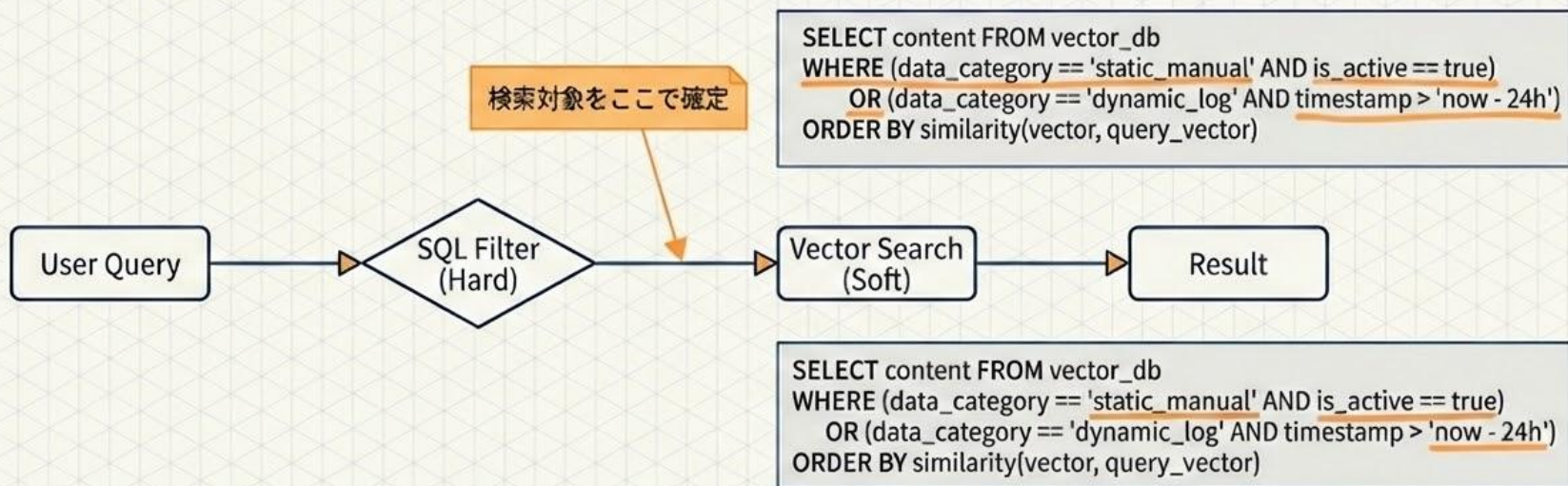
AIに全てのデータを見せてはいけない。検索の段階で、システム側が「正しい範囲」を強制的に指定する。

現場での効果:

「古いマニュアルと混ざってほしくない」という不安を、DB側のフィルタ機能で解決する。

3. 【検索制御】古い情報を遮断する「プレ・フィルタリング」

LLMに「毒」を食わせない：物理的な情報の遮断



仕組み:

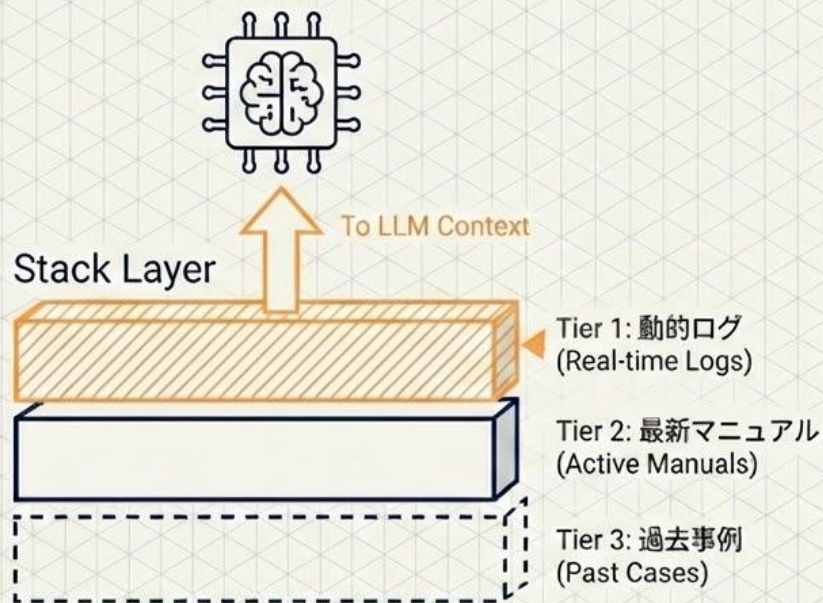
ベクトル検索（意味の近さ）を行う前に、メタデータで検索範囲を確定させる。

メリット:

AIが古い情報を「参照すること自体」ができなくなるため、ハルシネーションが構造的に発生しなくなる。

4. 【優先順位】情報の重み付け (Reranking)

情報の「鮮度」と「信頼度」でコンテキストを並び替える



スキーム: 多層コンテキスト・スタッキング

検索でヒットした情報を、LLMに渡す前に「重要度順」に並び替える。

優先順位:

- Tier 1 (最上段): 動的ログ (「今」起きている現場の異常)
- Tier 2 (中段): 最新マニュアル (守らなければならない不変のルール)
- Tier 3 (下段): 過去事例 (参考にするが、現在のルールと競合したら捨てる)

理由:

LLMは「最初に提示された情報」を強く意識する特性があるため。

5.【プロンプト】情報の「扱い方」を厳密に指示

情報の「優先順位」をAIに徹底させるシステムプロンプト

【役割】 あなたは工場の配置最適化を担当するAIです。

【情報の扱いに関する制約】

1. 提供された【現場ログ】に記載された「欠員」や「故障」を最優先の事実として扱うこと。
2. 【作業マニュアル】にある安全制約（例：単独作業禁止）は絶対に破らないこと。
3. 【過去事例】は、上記2点と矛盾しない範囲で「ヒント」としてのみ使用すること。

【エラーハンドリング】 情報が不足している場合は推測せず、「判断不可」と回答すること。

狙い：

複数の情報が矛盾した際、AIが「自分勝手な折衷案」を作るのを防ぐ。



AI Processor

6. 【根拠提示】「なぜそう判断したか」を数値で吐かせる

透明性の確保：エビデンス（根拠）付き回答の構造化

```
{
  "recommendation": "ラインAの作業員W002をラインCに移動",
  "reason": "ラインCの検品遅延を解消するため",
  "evidence": {
    "source_doc": "Manual_V1.2_P45",
    "log_timestamp": "2026-01-17T09:15:00",
    "confidence_score": 0.92,
    "citation_text": "ラインCの遅延時はラインAから応援を送る..."
  }
}
```

根拠オブジェクト

信頼度スコア
(数値化)



設計目標:

「答え」だけでは納品できない。その「根拠」をシステム側で検証可能にする。

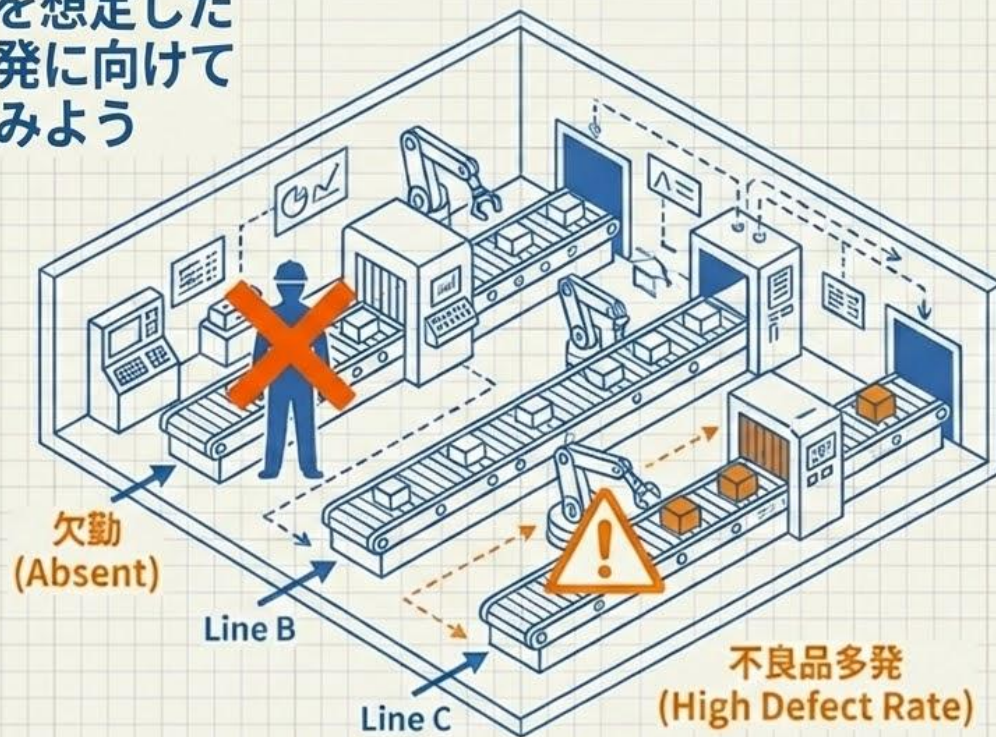


効果:

プログラム側で「信頼度が低い（0.8未満）」場合は、人間に確認を促すUIを表示できるようになる（Human-in-the-loop）。

ワークショップ：工場最適化ソリューション設計

実際の工場現場を想定した
ローカルLLM開発に向けて
RAGを構築してみよう



ミッション：工場現場の「カオス」をAIが理解できる「ロジック」へ変換せよ

非構造化データ (Raw Data)



- raw_manual.txt (Word：文章が長く条件が散乱)
- raw_daily_report.txt (手書き風：日付の曖昧さ)
- raw_shift_excel.csv (Excel：ノイズ混入)

Data Engineering

構造化データ (RAG-ready)



- 20_manual_knowledge.md (正規化されたルール)
- 21_production_log.json (計算済みのパラメータ)
- 22_resources.csv (クレンジング済み)

なぜ「そのままRAG」では失敗するのか？（3つの罠）

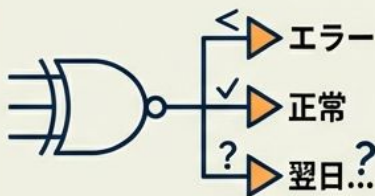
01. 曖昧性 (Ambiguity)



「今日」「明日」などの
相対日付。

RISK: AIは検索時点の日付と
文書内の日付を混同する。

02. 暗黙知 (Implicit Logic)



「エラーが出た翌日は...」
といった条件分岐。

RISK: 文脈を計算式として
認識できず、見落とす。

03. ノイズ (Noise)



「インフルエンザで...」
などの個人的理由。

RISK: 検索精度を下げ、ハル
シネーションの原因になる。

Engineering Philosophy: 計算と判断を分離する。確定的なロジックはデータ前処理で解決し、AIには「判断」のみを委ねる。

FILE 20: マニュアルの構造化設計 (Normalization)

長文テキストを「座標」で特定可能なマトリクスへ変換

Raw Text

ラインA (組立) の場合、
熟練者(A)は100個/h、
前回メンテから14日
経過すると... 罷めた
たちで、自話などの構節
熟練者のことに要しレ
ンタルで調整しても値段
が出される。

検索コスト高
条件が散在

Structured Table

ラインID	工程	ランク	能力
A	組立	A	100/h
A	組立	B	80/h

設計意図: 文章から「ラインAのBランク」を
探すのはコストが高い。テーブル化するこ
とで、AIは座標として数値を特定できる。

FILE 20: 条件式のコード化 (Logic Extraction)

Case 1: メンテナンス期間の閾値

Raw Text

「前回メンテから14日経過すると
効率が一律80%に落ちるそうです」

ノイズ (文末) を削除し、数値を抽出

条件: $(\text{今日の日付}) - (\text{最終メンテナンス日}) > 14\text{日}$
Action: 効率係数 = 0.8

Case 2: エラー対応の定量化

Raw Text

「エラー502 (ジャム) が出た翌日
は.....稼働率90%」

Trigger: Error 502
Action: 稼働率係数 = 0.9

自然言語の曖昧さを排除し、プログラミング言語に近い記法で定義する。

FILE 21: 日報のデジタル化とJSONパース (Digitization)

raw_daily_report.txt からのパラメータ抽出

1/20 日報...

~~~~~

Line C

不良90個...

Parsing &
Type Conversion

```
{  
  "date": "2025-01-20T00:00:00Z",  
  "line_id": "C",  
  "production_count": 2000,  
  "defect_count": 90  
}
```

- ✓ 日付型(ISO8601)への変換
- ✓ 数値型への変換 (文字列 '90' → 数値 90)

FILE 21: 隠れたアラートの顕在化 (Pre-calculation)

The Trap



日付の罠: 日報の日付は「1/20」だが、システム処理日は「1/21」の可能性がある。
「今日から何日前」の計算をAIに推論させてはいけない。

Engineering Solutions

経過日数の確定

Raw Text: Line Aのメンテ...今日で11日目



```
{  
  "days_since_maintenance": 16,  
  "maintenance_alert": true  
}
```

☒ マニュアルの14日ルールに基づきフラグを立てる

不良率の計算

Raw Text: 生産2000, 不良90



```
{  
  "defect_rate_percent": 4.5  
}
```

☒ マニュアルの「3%ルール」を超過している事実を、数値として明示する。

FILE 22: リソース情報のクレンジング (Noise Reduction)

raw_shift_excel.csv のノイズ除去と正規化

Before (Noisy)

Tanaka:	インフルエンザにより欠勤	
Suzuki:	明日(1/21)休みます	



After (Clean)

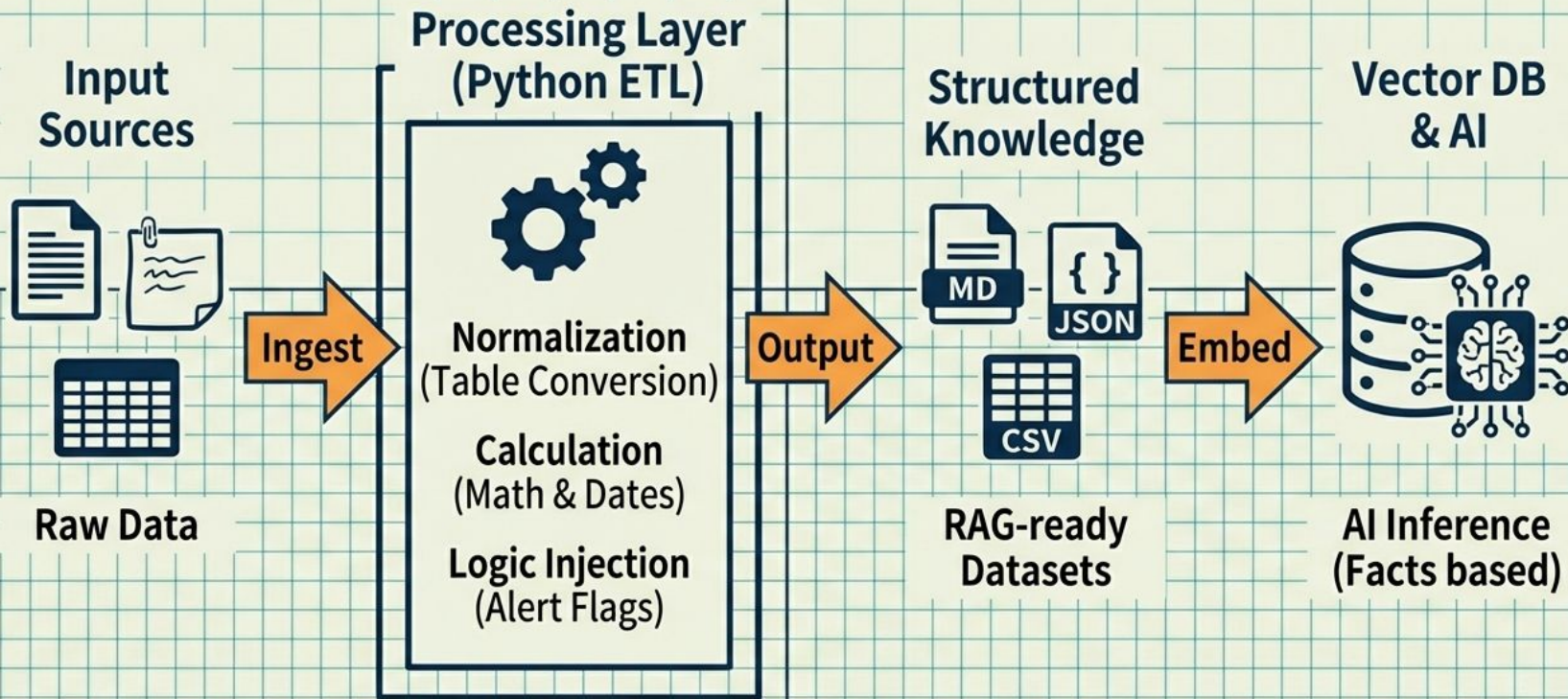
```
[  
  { "name": "Tanaka",  
    "status": "Absent",  
    "skills": [...] },  
  { "name": "Suzuki",  
    "status": "Absent",  
    "date": "2025-01-21" }  
]
```

個人的な理由はAIには不要なノイズ

Key Action Box

- ✓ Status Normalization: 理由は問わず "status": 'Absent' に統一。
- ✓ Structure: 将来的なスキル拡張（ネスト）に備え、フラットなCSVからJSON配列へ変換。

FILE 23: データエンジニアリング・パイプライン設計図



結論：なぜここまで加工するのか？

「計算 (Calculation)」と「推論 (Inference)」の分離

データエンジニアリング
(Deterministic)

- 正確な日数の計算
- 厳密な閾値判定 (3% rule)
- 日付の正規化



AI 推論
(Probabilistic)

- 文脈の理解
- 状況に基づく判断
- 提案の生成



確定的なロジックはデータ前処理 (Python) で解決する。
AIには「計算された事実 (Fact)」を渡し、高度な「判断」のみに集中させる。

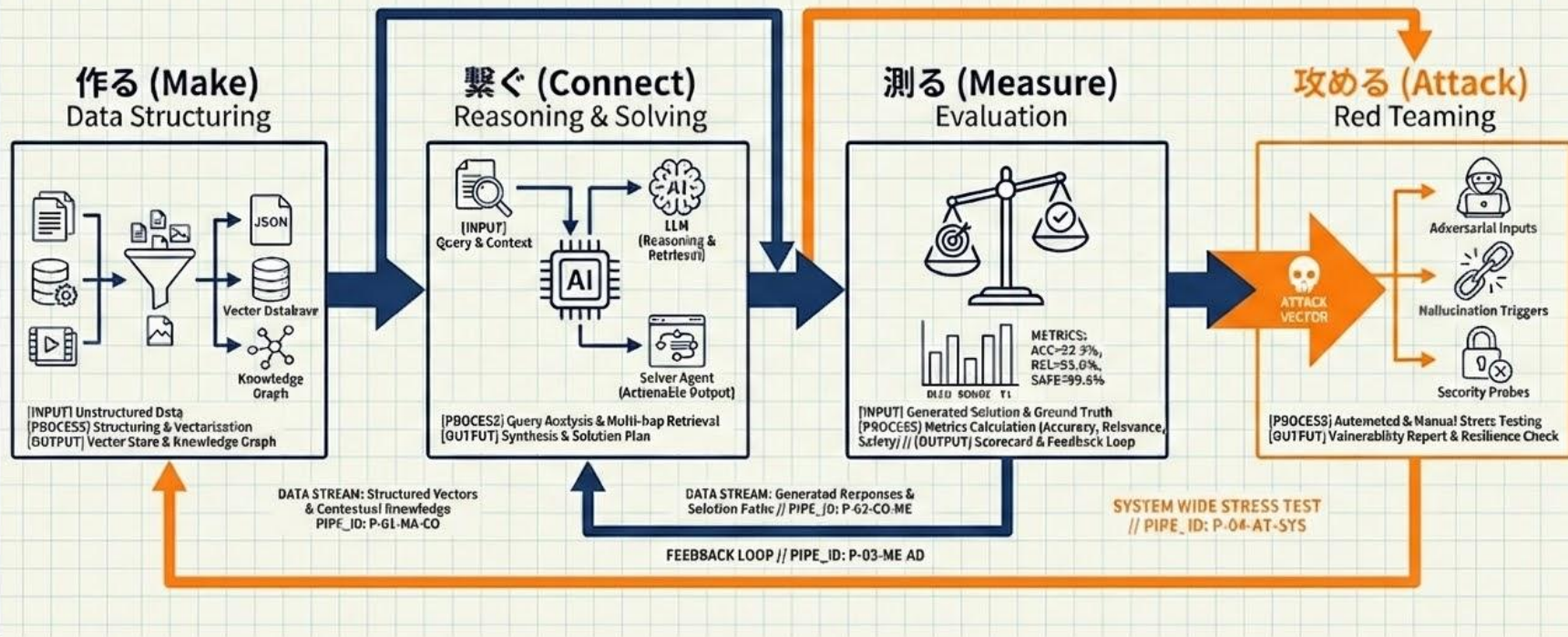
最終実装チェックリスト (Action Items)

- ☐ マニュアル: 文章をテーブル形式に正規化し、条件式 (if-then) を明確にしたか？
- ☐ 日報: 日付計算を済ませ、アラートフラグ (maintenance_alert) をJSONに付与したか？
- ☐ 計算: 不良率 (%) などの指標を事前に算出したか？
- ☐ リソース: 欠勤理由などのノイズを削除し、ステータスを統一したか？



実践的RAG構築プロンプト

RAGは単なる検索ではない。役割分担されたAIによる「自律的ワークフロー」である。

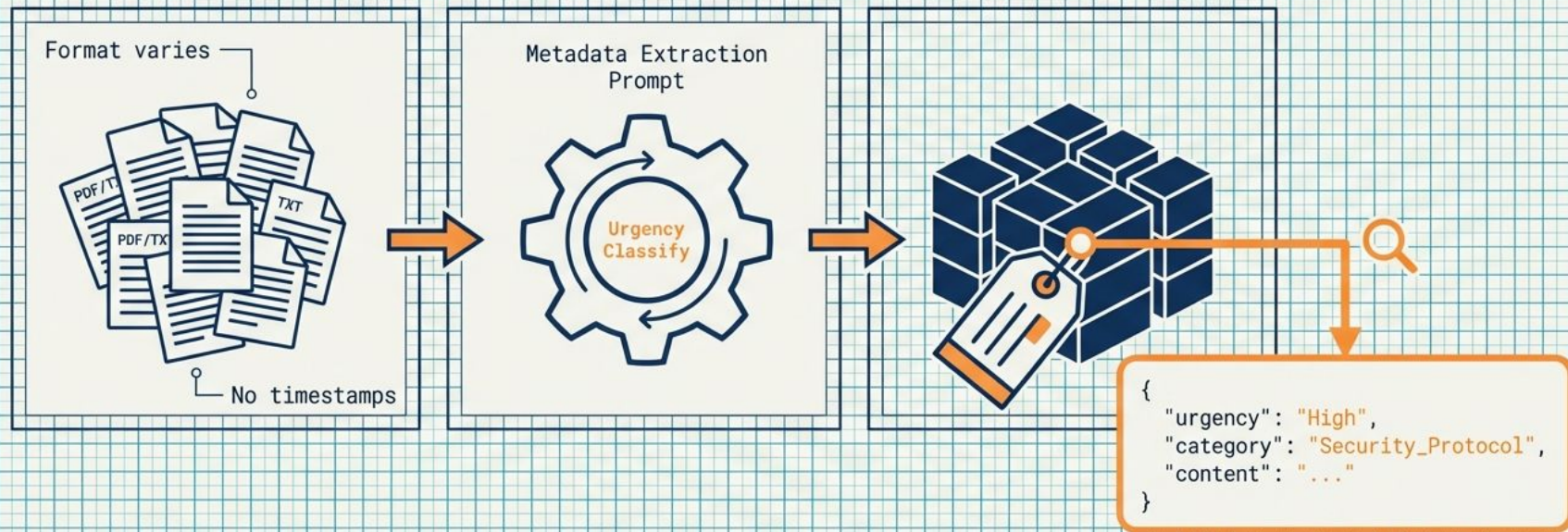


PROJECT: RAG_PIPELINE_V1 // STATUS: SYSTEM_OVERVIEW // MODULES: 04 // CONFIDENTIAL

Phase 1 【作るAI】 データを「検索できる形」にする

Problem:

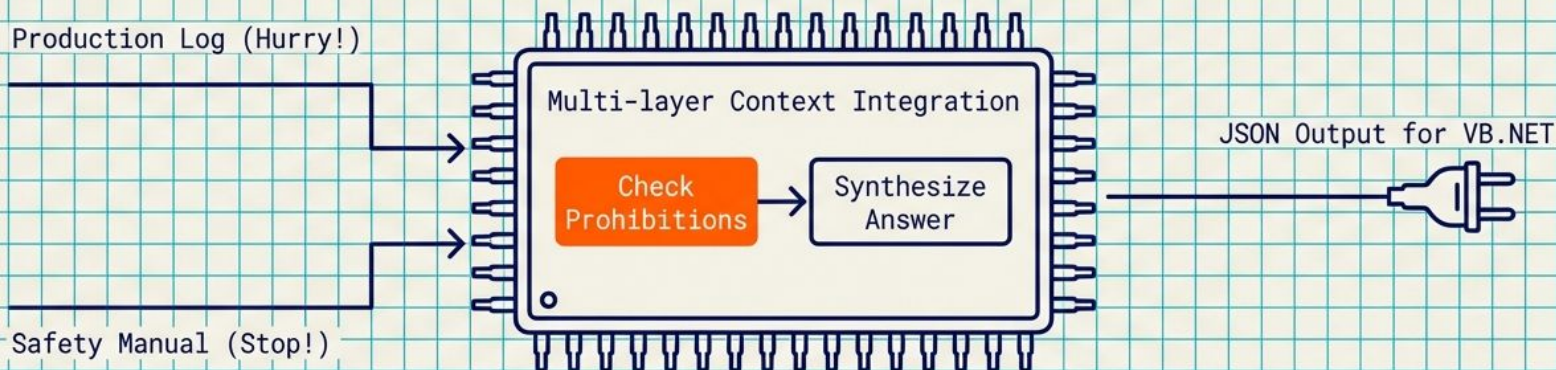
テキストのマニュアルをそのまま投入しても、AIは「いつのルールか」「緊急度は何か」を判断できない。



Benefit: 検索時に「緊急度Highのマニュアルだけ」といった高精度なフィルタリングが可能になる。

Phase 2【繋ぐAI】 矛盾する情報を「調停」する

⚠ Problem: 「生産を急げ」というログと、「安全を守れ」というマニュアル。AIはどう判断すべきか？



Mechanism:

人間の判断プロセス（思考）を模倣し、安全な回答のみを出力。

Phase 3 【測るAI】 AIの回答を「AIが採点」する

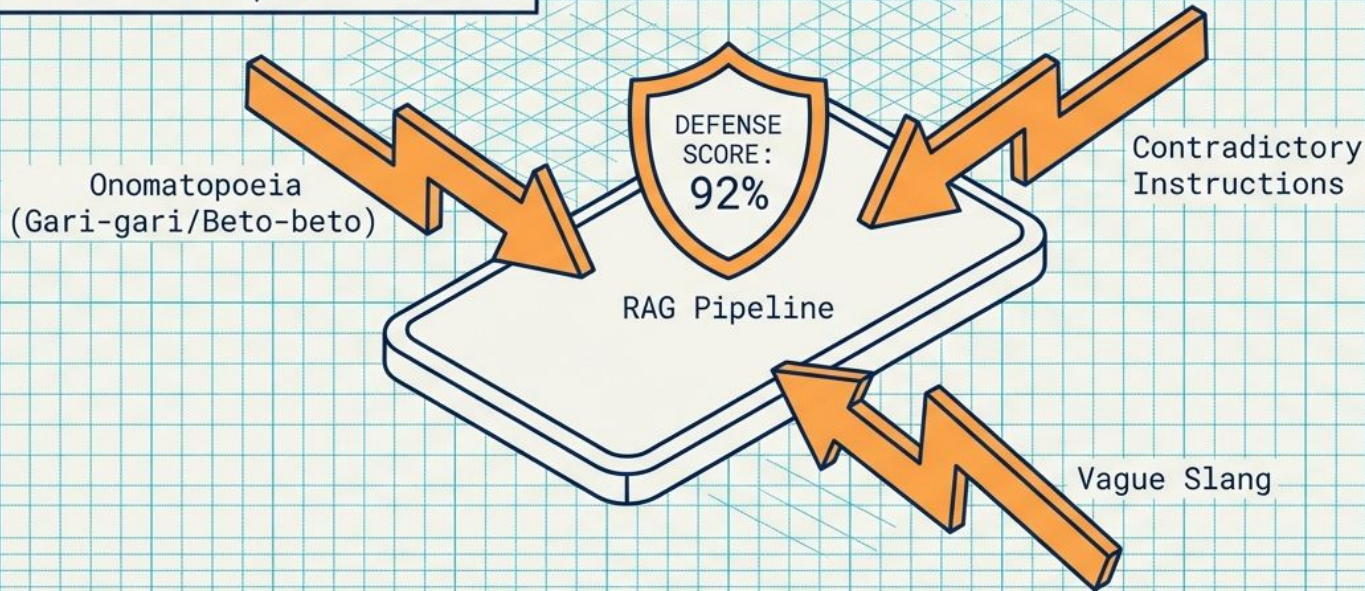
⚠ Problem: 人間による全数チェックは不可能。しかしAI任せにはリスクがある。



Benefit: 「全自動」を目指さず、「高精度な半自動」を構築することで信頼性を担保する。

Phase 4 【攻めるAI】 意地悪なテストで「穴」を塞ぐ

Stress Test: RAG Pipeline Defense



Red Teaming: 100パターンの意地悪な入力（エッジケース）を自動生成し、デプロイ前に弱点を特定する。

実装仕様書まとめ (Specification Summary)

BRAIN (脳)	FUNCTION (機能)	BUSINESS VALUE (価値)
MAKE (作る)	Meta-data Extraction	Value: Searchability (検索性)
CONNECT (繋ぐ)	Arbitration Logic	Value: Reliability (信頼性)
MEASURE (測る)	LLM-as-a-Judge	Value: Trust (品質担保)
ATTACK (攻める)	Red Teaming	Value: Robustness (堅牢性)

NEXT PHASE: MODEL BUILDING

Required: Python (PyTorch/TensorFlow) & GPU Computing

「質問を投げるな、ボタンを繋げ。」

RAGの限界と「脳の書き換え」

Knowledge vs Behavior

RAG (Retrieval)



外部知識の参照
(External Knowledge)

カンペを見る

思考パターンは
変わらない

Fine-Tuning



脳の最適化
(Internal Optimization)

脳を鍛える

振る舞いを変える

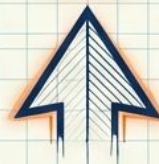
Key Insight: RAGは「知識」を補完し、ファインチューニングは「振る舞い」を最適化する。

次回、いよいよ「現場専用AI」が起動します

Aさんの現場データ (Data)
Noto Sans JP



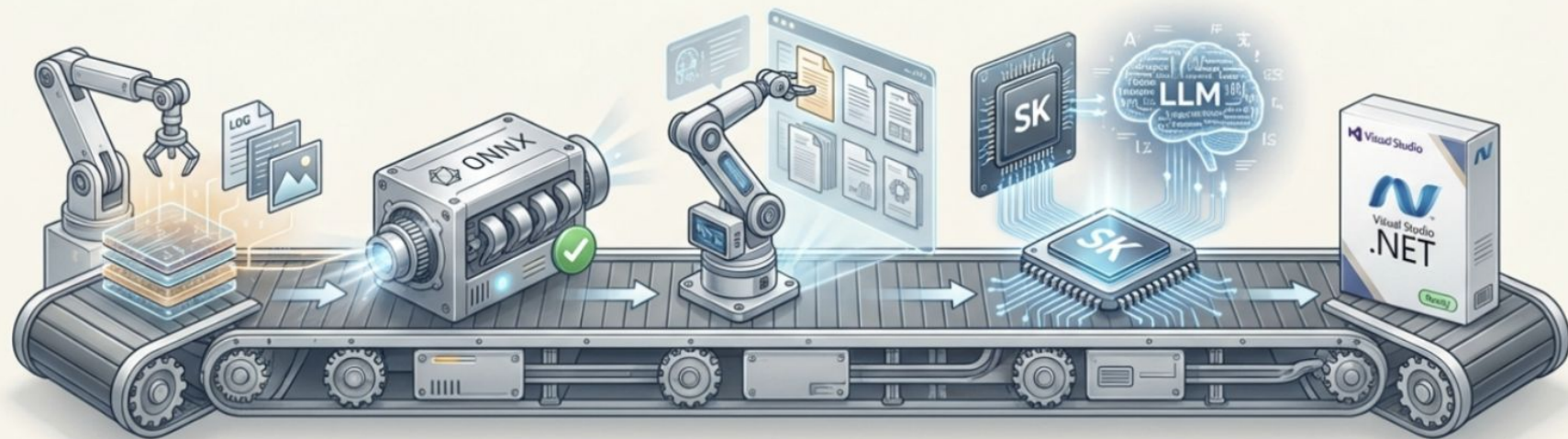
Bさんのローカル環境 (Env)
Roboto Mono



本日のRAG設計図
(Blueprint)

ネット不要・セキュア・超具体的。
現場が本当に欲しかった「自律型AI」を、自分たちの手で動かしましょう。

全体像：私たちが目指すAIシステムの組み立てライン



① 現場のデータ (原材料)

例：検査画像、ログファイル、
センサー値

② ONNXでAI判定 (精密エンジン)

OK/NG、異常ラベル、
確信度スコアなどを出力

③ RAGで社内 マニュアル検索 (部品カタログ)

(次回のテーマ) 判定結果に
関連する手順や仕様を検索

④ SK+LLMで 「わかる言葉」に変換 (中央制御・組立)

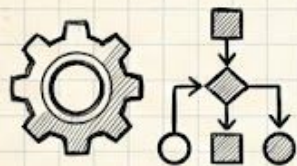
数値データを人間が理解で
きるレポート形式に整形

⑤ VB.NET/C# 業務アプリとして出荷 (完成品)

現場のPCで、誰でも使え
る形で提供

③ ロードマップ

最強の現場AI実現への3ステップ



今回

RAGの基礎とデータ設計
(Factory-Commander)

現場の知識をAIが使える形に
構造化する。

✓ 完了



次回

ローカルLLM構築実践
(セキュアな環境作り)

インターネット不要のAIサ
ーバーを自社PCに構築。

🕒 準備中



最終回

「独自課題解決」発表会

自分たちの現場の課題を、
RAG×ローカルLLMでどう解決
するか、プロトタイプを発表。

🚩 ゴール

【Mission1】現場の「秘伝のタレ」の構造化



Mission Parameters

- 1. Selection (選定)**
「ネットに上げたらクビになる」レベルの本物の現場資料（仕様書、日報、レガシーコード）を1つ選定する。
- 2. Structuring (構造化)**
Markdown形式（# Header）で整理する。
- 3. Tagging (付与)**
メタデータ（作成日時・部署・作成者）を冒頭に記載する。

Objective: 次回のローカルAIが、「汎用的な正解」ではなく「御社専用の正解」を導き出すための準備。

Mission 2: 脳 (Local LLM) の準備

あなたのPCを、インターネット不要のAIサーバーに変える『器』と『脳』の準備



【器】LM Studioのインストール

- 公式サイト (lmstudio.ai) からWindows版をダウンロード&インストール。



【脳】Llama 3.1 8Bのダウンロード

- 検索窓で「Llama 3.1 8B」を検索。
- 推奨モデル「Q4_K_M」（4ビット量子化）を選択。
- 注釈：「賢さと速度の黄金比。実務に最適！」



【起動】リソース管理とサーバー開始

- GPU Offloadを「Max」に設定。他のアプリを閉じてVRAMを確保。
- 「Start Server」をクリック。表示されるURLを確認。
- URL: ``http://localhost:1234/v1`` ← これが接続先！



Verification & Report

- テストチャットで日本語が通じれば合格。
- Aさんへの連絡：「器と脳の準備完了。次はRAGデータを！」